AI Critique Reflection Report

WalkingSG Use Case Diagram — Requirements Traceability Review

---

1. Original Prompt

Can you look at the functional and non-functional requirement in the SRS doc and help me critique the use case diagram image that is attached with respect to how well it matches the functional and non-functional requirement

(Submitted with two attachments: the WalkingSG Software Requirements Specification document, and an image of the WalkingSG UML use case diagram.)

2. Original AI Response

Quick verdict

Solid coverage of Authentication and most of Search/Routing/Navigation/Hazards, but there are a few gaps, one likely invented use case, and the diagram's structure (which use cases connect to which) doesn't reflect the actual sequential dependencies in the SRS.

Requirement-by-requirement coverage

SRS Section	Key Requirements	Diagram Match	Verdict
4.1 Authentication	Register, Login, Forgot Password, Logout, Edit/Delete Profile	RegisterAccount, Login, ResetPassword (extend Login), Logout, EditProfile, DeleteAccount (extend EditProfile)	Good — clean, correct extend usage
4.2 Preference Mgmt	One “Personal Declaration Form” in profile: mobility_needs, route_preference, toggles, age, UV sensitivity	CreateMobilityProfile, EditMobilityProfile — both floating, disconnected from EditProfile	Questionable
4.3 Search & Map	Render map, GPS permission/autofill, autocomplete via Search API, SG bounding-box validation	InitializeMap (+OneMap API), SearchLocation (+OneMap Search API)	Good actor mapping
4.4 Routing Engine	Generate 5 routes, apply weights/confidence filtering, sort, star top match	GeneratePersonalizedRoutes, <<include>> from SearchLocation	Correct include usage
(implied) route selection	“User selects a route” (from 4.5.2)	SelectPreferredRoute — but isolated, no link to GeneratePersonalizedRoutes or LiveNavigation	Orphaned use case
4.5 Live Navigation	GPS tracking, deviation/recalc, arrival detection, confidence +1	LiveNavigation (+GPS service)	Good
4.6 Hazards/Feed	Report segment as unwalkable during nav (-2 score), dedicated News screen (5km radius, sorted by recency)	ReportMapSegment <<extend>> LiveNavigation; ViewNearbyHazards (+GPS service)	Good, correct extend

SRS Section	Key Requirements	Diagram Match	Verdict
2.7.2 Dependencies	FCM/APNs push for “instant local hazard broadcasting”	Not modeled at all	Missing actor

Structural issues worth fixing

1. CreateMobilityProfile looks invented. Nothing in REQ-PREF-01 through 07 describes a separate “create” flow — the form just auto-fills with saved defaults (PREF-01.1) and new accounts get default values automatically (PREF-02.2, 03.2, 07.2). There's really just one profile-editing use case here, not two. I'd merge these into a single ManageMobilityPreferences and connect it to EditProfile (probably as <<include>>, since REQ-AUTH-05 already says “edit and manage profile”).

2. Floating use cases with no real dependency links. InitializeMap, SearchLocation, SelectPreferredRoute, and LiveNavigation are all drawn as independent branches off User, but the SRS stimulus/response sequences make them clearly sequential (map must init → search → routes generated → user selects → navigation starts). Right now SelectPreferredRoute doesn't connect to anything, which makes it look like a dead-end use case. Worth adding <<include>> from LiveNavigation to SelectPreferredRoute (you can't navigate without having selected a route first).

3. OneMap API split may not match the SRS. Section 3.3 describes one “OneMap API” providing autocomplete + base map + barrier-free routing data. You've split it into “OneMap Search API” (→ SearchLocation) and “OneMap API” (→ InitializeMap only). If you keep the split, GeneratePersonalizedRoutes should probably also connect to the OneMap API actor, since REQ-RTE-01.3 (Barrier-Free routing) and REQ-PREF-03 (confidence-score filtering) depend on that same baseline routing data — right now the routing engine looks like it operates without any external data source.

4. Missing push notification actor. The hazard feed is described as “instant broadcasting” via FCM/APNs to nearby users — that's a push, not just a pull. Your ViewNearbyHazards is modeled as a pure pull interaction (user opens screen, queries GPS-filtered hazards). If the push behavior matters to your grading rubric, you're missing an actor/use case for it.

On non-functional requirements

Your diagram doesn't reflect any NFRs (performance, security, safety, quality attributes) — that's actually normal, UCDs aren't meant to carry NFRs directly. If your rubric wants NFR traceability shown visually, the common trick is attaching UML notes to relevant use cases (e.g., a note on LiveNavigation for the “auto-hide screen above 15km/h” safety rule, or on Login/EditProfile for password hashing + 30-day session expiry). Not required, but easy points if your grading criteria mention NFR traceability.

TL;DR fixes

1. Merge CreateMobilityProfile + EditMobilityProfile → one use case, link to EditProfile
2. Connect SelectPreferredRoute into the flow (include from LiveNavigation)
3. Reconsider the OneMap actor split / connect it to routing
4. Add a note or actor for the FCM/APNs push mechanism if it's in scope
5. Optionally add NFR notes on key use cases for traceability

3. Selected Aspect of the Critique

Quote:

> *“(implied) route selection — ‘User selects a route’ (from 4.5.2) — SelectPreferredRoute — but isolated, no link to GeneratePersonalizedRoutes or LiveNavigation — Orphaned use case”*

4. Justification

It would make sense that our 5 route generator should be a pre-requisite to the SelectedPreferredRoute. Just as LiveNavigation cannot start without a route already been selected. It could mislead the reader of the diagram into thinking route selection is optional or unrelated feature rather than a mandatory step between route generation and navigation.